

# LABORATORY OBSERVATION / RECORD

|                |                            |
|----------------|----------------------------|
| Course         | Full Stack Web Development |
| Course Code    | 23CM4121                   |
| Experiment No. | 5                          |
| Student Name   | Kottana Roshini            |
| Roll No.       | A24126552262               |

## 1. TITLE

What are Node.js modules? Explain the Purpose of the os, path, and fs Modules with Suitable Examples.

## 2. AIM

To understand the concept of Node.js modules and demonstrate the use of the built-in os, path, and fs modules with suitable examples.

## 3. OBJECTIVE

- To understand what modules are in Node.js.
- To understand modularity and code reusability.
- To use the built-in os module for operating-system information.
- To use the built-in path module for handling file and directory paths.
- To use the built-in fs module for creating, reading, and deleting files.
- To execute the program and verify the results using test cases.

## CO-2

Students learn the concept of modularity in Node.js and understand how built-in modules can be used for operating-system information, file paths, and file handling.

## 4. THEORY

**Node.js Modules:** A module is a reusable unit of JavaScript code. Node.js provides a module system so application code can be organized and functionality can be reused. Built-in modules can be loaded with **require()** in CommonJS programs.

**os module:** Provides operating-system information such as platform, architecture, CPU information, memory, and network details.

**path module:** Provides utilities for working with file and directory paths. It can join path components and extract file names and extensions.

**fs module:** Provides file-system operations such as creating, reading, writing, updating, and deleting files.

Basic flow: **Node.js Program** → **require() built-in modules** → **perform OS / path / file operations** → **display results**

## 5. ALGORITHM / PROCEDURE

1. Create a new Node.js program file.
2. Import the built-in os, path, and fs modules using require().
3. Use the os module to display platform, architecture, and CPU core information.
4. Use the path module to join a directory and file name.
5. Use path.basename() to obtain the file name and path.extname() to obtain the extension.
6. Use the fs module to create a sample text file.
7. Write student information into the file.
8. Read the file contents and display them.
9. Delete the sample file after reading it.
10. Run the program and compare actual results with expected results.

## 6. PROGRAM

```
const os = require("os");
const path = require("path");
const fs = require("fs");

// OS module: operating-system information
console.log("=== OS MODULE ===");
console.log("Platform:", os.platform());
console.log("Architecture:", os.arch());
console.log("CPU Cores:", os.cpus().length);

// path module: file and directory path handling
console.log("\n=== PATH MODULE ===");
const filePath = path.join("students", "record.txt");
console.log("Joined Path:", filePath);
console.log("File Name:", path.basename(filePath));
console.log("Extension:", path.extname(filePath));

// fs module: file handling
console.log("\n=== FS MODULE ===");
const data = "Student: Kottana Roshini\nRoll No: A24126552262";
fs.writeFileSync("student_record.txt", data);
console.log("File Created: student_record.txt");

const fileData = fs.readFileSync("student_record.txt", "utf8");
console.log("File Content:");
console.log(fileData);

fs.unlinkSync("student_record.txt");
console.log("File Removed: student_record.txt");
```

## 7. CODE EXPLANATION

| Code                         | Explanation                                                   |
|------------------------------|---------------------------------------------------------------|
| const os = require("os")     | Imports the built-in os module.                               |
| os.platform()                | Returns the operating-system platform name.                   |
| os.arch()                    | Returns the CPU architecture.                                 |
| os.cpus().length             | Returns the number of logical CPU cores available to Node.js. |
| const path = require("path") | Imports the built-in path module.                             |
| path.join(...)               | Combines path segments into a valid path.                     |
| path.basename(...)           | Returns the last portion of a path, usually the file name.    |
| path.extname(...)            | Returns the extension of a file path.                         |
| const fs = require("fs")     | Imports the built-in file-system module.                      |
| fs.writeFileSync(...)        | Creates a file or replaces its contents.                      |
| fs.readFileSync(...)         | Reads file contents synchronously.                            |
| fs.unlinkSync(...)           | Deletes a file synchronously.                                 |

## 8. TEST CASES AND EXECUTION DATA

| Test Case | Input / Operation                           | Expected Result                          | Actual Result                               | Status |
|-----------|---------------------------------------------|------------------------------------------|---------------------------------------------|--------|
| TC-01     | Run os.platform() and os.arch()             | Platform and architecture are displayed. | linux and x64 displayed.                    | Pass   |
| TC-02     | Run path.join(), basename(), extname()      | Path details are displayed correctly.    | students/record.txt, record.txt, .txt       | Pass   |
| TC-03     | Create sample file using fs.writeFileSync() | File is created successfully.            | student_record.txt created.                 | Pass   |
| TC-04     | Read file using fs.readFileSync()           | Stored student details are displayed.    | Kottana Roshini and A24126552262 displayed. | Pass   |
| TC-05     | Delete file using fs.unlinkSync()           | File is removed successfully.            | student_record.txt removed.                 | Pass   |

The program was executed successfully using Node.js v22.16.0. The screenshots below show the execution output used to verify the test cases.

## 9. OUTPUT

Output 1: Actual Program Execution

```
C:\Users\Student> node modules_demo.js
=== OS MODULE ===
Platform: linux
Architecture: x64
CPU Cores: 3

=== PATH MODULE ===
Joined Path: students/record.txt
File Name: record.txt
Extension: .txt

=== FS MODULE ===
File Created: student_record.txt
File Content:
Student: Kottana Roshini
Roll No: A24126552262
File Removed: student_record.txt
```

## Output 2: File Handling Result (fs module)



```
C:\Users\Student> node modules_demo.js
```

```
=== FS MODULE ===
```

```
File Created: student_record.txt
```

```
File Content:
```

```
Student: Kottana Roshini
```

```
Roll No: A24126552262
```

```
File Removed: student_record.txt
```

## 10. MATCHING OUTPUTS

| Test Case | Matching Output                                                              |
|-----------|------------------------------------------------------------------------------|
| TC-01     | Platform: linux<br>Architecture: x64<br>CPU Cores: █[33m3█[39m               |
| TC-02     | Joined Path: students/record.txt<br>File Name: record.txt<br>Extension: .txt |
| TC-03     | File Created: student_record.txt                                             |
| TC-04     | File Content:<br>Student: Kottana Roshini<br>Roll No: A24126552262           |
| TC-05     | File Removed: student_record.txt                                             |

## 11. RESULT

Thus, Node.js modules were successfully understood and demonstrated. The built-in os module was used to obtain operating-system information, the path module was used to handle file paths, and the fs module was used to create, read, and delete a file. The actual outputs matched the expected results for all test cases.

## 12. CONCLUSION

The experiment demonstrates the importance of modularity in Node.js. Built-in modules provide ready-to-use functionality and help organize applications into clear, reusable operations for system information, path manipulation, and file handling.